Exam Kit: Lectures 1 to 4

Introduction to compilers and lexical analysis. A 100-point practice exam, with full solutions under every question

How to use this kit

This is a full practice exam on Lectures 1 to 4 only (introduction to compilers; tokens and regular expressions; automating lexical analysis; lexical analysis in practice). Nothing from the parsing lecture appears. The weights copy the style of the course's mock exams: a long multiple-choice part where every answer must be justified in one line, short questions on the compiler pipeline, construction problems on automata and lexers, and a true/false sweep.

Part	Content	Points
A	20 multiple-choice questions, one line of justification each	40
B	5 short questions on the compiler pipeline (Lecture 1)	15
C	12 construction problems on regular expressions, automata and lexers (Lectures 2 to 4)	35
D	20 true/false statements	10
	Suggested time: 2 hours 30 minutes	100

Each solution sits directly under its question. Every automaton, scan trace, regular-expression equivalence and program output in this kit was checked by program (`check_lex.py`, `flex` and `cc`). Conventions: ε is the empty string, `D` stands for `[0-9]`, `L` for `[a-zA-Z]`, and Thompson counts use **convention B** (concatenation merges two states) unless a question says otherwise.

Part A: Multiple choice (20 × 2 points)

Choose one answer and justify it in one line. A correct letter without a justification earns one point.

A1. Compiled or interpreted?

`javac` turns `.java` files into `.class` files, which the JVM executes. Which statement is accurate?

(A) Java is an interpreted language, so `javac` is not a compiler.
(B) `javac` is a compiler whose target language is JVM bytecode; the JVM then interprets that bytecode and JIT-compiles the hot parts to native code.
(C) `javac` produces native machine code for the machine it runs on.
(D) The JVM is an assembler.

Solution

(B). A compiler translates from a source language to a target language preserving meaning; the target need not be native code. CPython does the same with its bytecode. "Compiled versus interpreted" is a property of an implementation, not of a language.

A2. When to compile

A 200-line data-cleaning script will be run exactly once, over a 50 GB input. Which choice is best justified?

(A) Interpret it: it runs once, and compile time would be wasted.

(B) Compile it (or JIT it): the single run is enormous, so execution time dominates compile time by far.
(C) Interpret it: interpreters are always faster on large inputs.
(D) Neither matters: both take the same total time.

Solution

(B). The rule of thumb "compile when you run many times, interpret when once" is really about total execution time versus compile time. Here one run does billions of operations, so a few seconds of compilation (or a JIT, which compiles the hot loops while the program runs) are repaid immediately.

A3. Front ends and back ends

A vendor supports 4 source languages on 3 target machines. With a shared intermediate representation the number of translator components to write is
(A) 12 (B) 7 (C) 4 (D) 3

Solution

(B). $m + n = 4 + 3$: one front end per language emitting the IR, one back end per machine consuming it. Without the IR it would be $m \times n = 12$ full compilers.

A4. Which phase?

The statement `int n = "seven";` is rejected by
(A) the lexer (B) the parser (C) semantic analysis (D) the linker

Solution

(C). Every token is legal and the statement has the shape *type identifier = expression*, so lexing and parsing succeed. Assigning a string to an `int` is a type error, found by the tree walk of semantic analysis.

A5. Whose message is it?

`undefined reference to 'sqrt'` is reported by
(A) the lexer (B) semantic analysis (C) the assembler (D) the linker

Solution

(D). The compiler saw a declaration of `sqrt` (from a header) and emitted a call to an external symbol. Resolving that symbol across object files and libraries is the linker's job; it fails when no object file defines it (here `-lm` was missing).

A6. Parentheses

In `(a + b) * c`, the parentheses are
(A) kept as nodes of the AST so that code generation knows the order;
(B) discarded by the lexer as whitespace;
(C) used by the parser to shape the tree, then dropped, because the tree shape already encodes the grouping;
(D) replaced by temporaries during optimisation.

Solution

(C). They are tokens (not whitespace), and the parser needs them to put `+` below `*`. Once the AST has that shape, the parentheses carry no information.

A7. The most important optimisation

Lecture 1's `findIndex` example went from 18 memory accesses per iteration to none. The transformation responsible is

(A) constant folding (B) register allocation (C) dead-code elimination (D) inlining

Solution

(B). Naive code keeps every variable in memory and loads/stores it at each use. Register allocation keeps `i`, `n` and the array pointer in registers for the whole loop.

A8. The Chomsky hierarchy and the phases

Which pairing is wrong?

(A) Lexing: regular languages, finite automata.
(B) Parsing: context-free languages, pushdown automata.
(C) "Every variable is declared before use": context-free, so it is checked by the parser.
(D) "This loop terminates": undecidable in general, so the optimiser approximates conservatively.

Solution

(C) is wrong. Declaration-before-use is a context-sensitive constraint (it relates two arbitrarily distant places in the text and the set of names is unbounded); it is checked by semantic analysis with a symbol table, not by the grammar.

A9. Dead code

An optimiser is asked to delete unreachable statements. Because reachability is undecidable, a correct optimiser

(A) refuses to delete anything;
(B) deletes a statement only when it can *prove* it unreachable, and otherwise keeps it;
(C) deletes a statement unless it can prove it reachable;
(D) runs the program to find out.

Solution

(B). Soundness first: a wrong deletion changes the program's meaning, a missed deletion only costs speed. So the optimiser computes an over-approximation of the reachable set and deletes only what lies outside it.

A10. Tokens and lexemes

For `count = count + 1`, with the usual rules, which statement is true?

(A) There are 5 tokens and the two occurrences of `count` are the same token with the same attribute.
(B) There are 3 tokens, because `count` appears twice.
(C) There are 5 tokens, and the lexer already decides that both `count`s refer to the same variable.
(D) There are 7 tokens, counting the spaces.

Solution

(A). Five tokens: `ID(count) ASSIGN ID(count) PLUS NUM(1)`; spaces are discarded. Both `count`s are `ID` with lexeme `count`; whether they denote the same variable is a scoping question for semantic analysis, not the lexer (so (C) is wrong).

A11. A regular expression

Which expression denotes exactly the strings over $\{a, b\}$ with an *even* number of a's?
(A) $(aa \mid b)^*$ (B) $(b \mid ab^*a)^*$ (C) $b^*(ab^*a)^*$ (D) $(a \mid b)^*aa(a \mid b)^*$

Solution

(B). Read the string as blocks: either a b, or an a followed by some b's and the matching second a. (A) rejects aba (two a's separated by a b). (C) rejects aab (a b after the last block). (D) requires aa to appear, so it rejects ε and aba and accepts aaa.

A12. Counting Thompson states

Under convention B, Thompson's construction for $(a \mid b)^*abb$ produces
(A) 9 states (B) 10 states (C) 11 states (D) 14 states

Solution

(C). Leaves 5, unions 1, stars 1, concatenations 3: $2(5 + 1 + 1) - 3 = 11$. Convention A (a fresh ε-edge per concatenation) gives 14, answer (D), which is why the convention must be stated.

A13. Subset construction

An NFA with n states is converted to a DFA by the subset construction. Which is true?
(A) The DFA always has at most n states.
(B) The DFA can have up to 2^n states, and some languages ("the k-th symbol from the end is a") really need 2^k.
(C) The DFA always has exactly 2^n states.
(D) The blow-up disappears after minimisation.

Solution

(B). States are subsets, so 2^n is the ceiling; usually far fewer are reachable. For L_k the 2^k strings of length k are pairwise distinguishable (Method 7), so even the minimal DFA has 2^k states: (D) is false.

A14. Minimal DFAs

Two students minimise the same DFA and obtain automata with different state names but the same number of states. Which is true?
(A) At most one of them is right.
(B) Both can be right: the minimal DFA is unique up to renaming of states.
(C) Both are wrong, since minimisation always yields fewer states than the NFA.
(D) Minimal DFAs are not unique, so nothing can be concluded.

Solution

(B). Myhill–Nerode: the minimal DFA's states correspond to the distinguishability classes of the language, so any two minimal DFAs are isomorphic. (C) is nonsense: a minimal DFA can have more states than the NFA.

A15. Longest match

Rules, in order: `IF = if`, `IN = in`, `INT = int`, `ID = L(L|D)*`. The input `integer` is lexed as
(A) `IN ID(teger)` (B) `INT ID(eger)` (C) `ID(integer)` (D) `IF` then an error

Solution

(C). Longest match comes before rule order: `ID` matches all 7 characters, the keywords match 2 or 3. Rule order only breaks *ties* of equal length.

A16. Rule order

With the same rules, `int` is lexed as `INT` and not `ID`, because
(A) keywords are always preferred to identifiers by every lexer generator;
(B) both rules match 3 characters, and the tie is broken by the earlier rule;
(C) `ID` cannot match `int`;
(D) `INT` is longer than `ID`.

Solution

(B). `ID` does match `int`; the tie goes to the first rule listed. Put `ID` first and every keyword disappears (the generator warns "rule cannot be matched").

A17. The dot

In `flex`, a rule `"//".*` for line comments works because
(A) `.` matches any character including newline, and the comment ends at end of file;
(B) `.` matches any character *except* newline, so the match stops at the end of the line;
(C) `*` is non-greedy by default;
(D) `//` is a special token for comments.

Solution

(B). The same rule with `[^x]*` (which does include newline) would eat the rest of the file. For block comments, `"/*".*"*/"` is wrong for a different reason: it runs to the *last* `*/` on the line.

A18. The classic hand-written bug

A hand-written lexer reads identifiers with `while (isIdChar(c = read()))`. On `sum+1` it produces `ID(sum) NUM(1)`. The cause is
(A) `+` is not a token in this language;
(B) the loop consumed the `+` to discover that the identifier was over, and never gave it back;
(C) `isIdChar` is wrong;
(D) longest match prefers `NUM`.

Solution

(B). One character of lookahead was consumed instead of peeked. The fix is `peek()` before `read()`, or `ungetc` the terminator. The bug hides when the terminator is a blank.

A19. Nested comments

Properly nested block comments cannot be recognised by a regular expression because
(A) comments may contain newlines;
(B) a finite automaton cannot count an unbounded nesting depth; the language is the shape of balanced parentheses;

(C) `/*` contains a metacharacter;
(D) they can, with `"/*"(.|\n)*"*/"`.

Solution

(B). With N states, depths $0, \ldots, N$ cannot all be told apart (pigeonhole), so some depth would be closed too early. Lexer generators handle it with a depth counter in the action. (D) runs to the last `*/` of the file.

A20. From source to a running program

Put in order the tools that turn `main.c` and `util.c` into a running process, and name what resolves the call from `main` to a function defined in `util.c`.

(A) compiler, linker, assembler, loader; the assembler.
(B) compiler, assembler, linker, loader; the linker.
(C) assembler, compiler, loader, linker; the loader.
(D) compiler, assembler, loader; the compiler, since it sees both files.

Solution

(B). Each `.c` is compiled to assembly text and assembled to an object file with unresolved external symbols. The linker joins the object files and fills in cross-file addresses; the loader maps the executable into memory. The compiler sees one translation unit at a time, so (D) is wrong.

Part B: The compiler pipeline (5 × 3 points)

B1. Sort the errors

For each message, name the stage that produces it (lexer, parser, semantic analysis, linker, run time) and say in a few words why that stage and not an earlier one.

1. `x = y +* 2;`
2. `"unterminated string`
3. `return 3 # 4;`
4. `undeclared identifier ‘tmp’`
5. `undefined reference to ‘parse_header’`
6. `integer division by zero`

Solution

1. **Parser.** Every token is fine (`+` and `*` are both operators); no grammar rule allows two binary operators in a row.
2. **Lexer.** A string literal must close before the end of the line; no token can be formed from the opening quote onwards.
3. **Lexer.** `#` is not the start of any token (outside the preprocessor); it is a lexical error, before any grammar is involved.
4. **Semantic analysis.** The parser accepts any identifier; only the symbol-table walk knows that `tmp` was never declared.
5. **Linker.** The compiler saw a declaration and emitted a call to an external symbol; no object file or library defines it.
6. **Run time.** Whether the divisor is zero depends on the input in general (undecidable at compile time), so it is detected when the instruction executes.

B2. The intermediate representation

A company maintains compilers for C, Rust and Swift targeting x86, ARM and RISC-V, built around one shared IR.

(a) How many front ends and back ends exist, and what would the count be without the IR?

(b) A new optimisation (loop unrolling) is to be added. Where in the pipeline does it go, and how many copies of it must be written? Why?

(c) The company adds WebAssembly as a target. What has to be written, and what does not?

Solution

(a) 3 front ends and 3 back ends, 6 components; without the IR, $3 \times 3 = 9$ complete compilers. **(b)** In the middle, as an IR-to-IR pass, written *once*: it then serves every language and every target, because the IR is the shared contract between the two halves. (An optimisation written on the C AST would help only C; one written on x86 assembly only x86.) **(c)** One new back end (IR to WebAssembly: instruction selection, register/stack mapping, output format). No front end and no middle-end optimisation changes.

B3. Compile, interpret, or both

For each situation say whether you would compile ahead of time, interpret, or use a JIT, and give the deciding reason in one sentence.

(a) A spreadsheet formula typed by a user and evaluated once.

(b) A game engine's physics loop shipped to millions of players.

(c) A web browser's JavaScript engine running an unknown page.

Solution

(a) Interpret: it runs once and the formula is tiny, so any compile time exceeds the evaluation. **(b)** Compile ahead of time: the loop runs billions of times, and the compile cost is paid once by the vendor; the player never waits. **(c)** JIT: the code is unknown until it arrives and much of it runs once (interpret it), but hot loops are detected at run time and compiled to native code. The JVM makes the same choice.

B4. Where in the hierarchy

Complete the table: for each property, give the Chomsky type (or "undecidable"), the machine that recognises it, and the compiler phase that deals with it.

Property or language	Type	Machine	Phase
Floating-point literals			
Properly nested `begin`/`end` blocks			
Every `break` is inside a loop			
"This function never returns `null`"			

Solution

Property	Type	Machine	Phase
Floating-point literals	3 (regular)	finite automaton	lexer
Nested `begin`/`end`	2 (context-free)	pushdown automaton	parser
`break` inside a loop	context-sensitive	tree walk with context	semantic analysis
Never returns `null`	undecidable (Rice)	none; conservative approximation	optimiser (static analysis)

"`break` inside a loop" could be forced into a grammar by duplicating every statement rule in an in-loop and out-of-loop version, but that is exactly the kind of context the grammar should not carry; one tree walk with a flag does it.

B5. Looking inside the pipeline

Match each command or tool to what it produces, and name the stage where `undefined reference` and `expected ';'` are reported.

`gcc -E` `gcc -S` `gcc -c` `as` `ld` `objdump -d`

Solution

`gcc -E`: preprocessed source (macros expanded, headers pasted). `gcc -S`: assembly *text*, the compiler's real output. `gcc -c`: object file, bytes with a symbol table and unresolved references (compiler + assembler). `as`: the assembler alone, assembly to object. `ld`: the linker, object files and libraries to an executable. `objdump -d`: disassembles object or executable bytes back to readable assembly. `expected ';'` is the parser's (during `-S`); `undefined reference` is `ld`'s.

Part C: Constructions (35 points)

C1. Write the regular expressions (4 points) ($\star$)

Give a regular expression for each token. Use `D` for a digit, `L` for a letter, and ordinary regex operators.

(a) C identifiers: letters, digits and underscores, not starting with a digit.

(b) Hexadecimal literals such as `0x1F`, `0XdeadBEEF`, with at least one hex digit.

(c) Decimal literals with a point, where at least one side of the point has a digit (`3.`, `.5`, `3.5` but not `.`), and an optional exponent `e-12`, `E7`.

(d) String literals: double quotes around any characters, where `\"` is an escaped quote and `\\` an escaped backslash, and a newline is not allowed inside.

(e) Binary strings with an even number of `1`s.

(f) Strings over $\{a, b\}$ that never contain two consecutive b's.

Solution

(a) `(L|_)(L|D|_)*`
(b) `0(x|X)(D|[a-f]|[A-F])+`
(c) `(D+.D*|.D+)((e|E)(+|-)?D+)?`. The two alternatives before the exponent are what forbid a bare point. Writing `D*.D*` would accept `.` alone.
(d) `"([^"\\\n]|\\.)*"`: a body of ordinary characters (not quote, not backslash, not newline) or a backslash followed by any one character. Allowing `\\` as an escape is what makes `"a\\"` end at the second quote.
(e) `(0|10*1)*`. Blocks: a `0`, or a `1` followed by zeros and its partner `1`. (Checked against all strings up to length 6.)
(f) `(a|ba)*(b|ε)`, equivalently `(b|ε)(a|ab)*`. Every b is glued to an a except possibly the last character. (Checked up to length 7.)

C2. Token sequence with longest match (3 points) (⋆⋆)

Rules in priority order: `IF = if`, `LE = <=`, `EQ = ==`, `LT = <`, `ASSIGN = =`, `FLOAT = D+.D+`, `NUM = D+`, `ID = L(L|D)*`. Whitespace separates and is dropped; a character that starts no token is reported as an error and skipped.

(a) Give the token sequence for `if1<=x2=3.14.5<==y` (no spaces in the input).

(b) For `iff<==3.<=x`, give the sequence and say at which token the scanner has to push back *two* characters.

Solution

(a) `ID(if1) LE ID(x2) ASSIGN FLOAT(3.14) error(.) NUM(5) LE ASSIGN ID(y)`.
`if1`: `ID` matches 3 characters, `IF` only 2, so longest match wins. `<=`: both `LE` and `LT` match a prefix; `LE` is longer. `3.14`: `FLOAT` beats `NUM`. Then `.5`: no rule starts with a point, so `.` is an error and `5` is a `NUM`. `<==`: longest match from the first `<` is `<=` (there is no `<==` rule), then `=` alone is `ASSIGN`, not `EQ`.
(b) `ID(iff) LE ASSIGN NUM(3) error(.) LE ID(x)`. At `3.<`: after `3` the scanner is in an accepting state for `NUM`, reads `.` (still alive for `FLOAT`), reads `<` (dead), and must push back `.<`, two characters, then emit `NUM(3)`. The `.` then fails on its own. Total reads for the 11 characters: 17.

C3. Thompson's construction (3 points) (⋆⋆)

Let $r = (ab \mid b)^* a$.

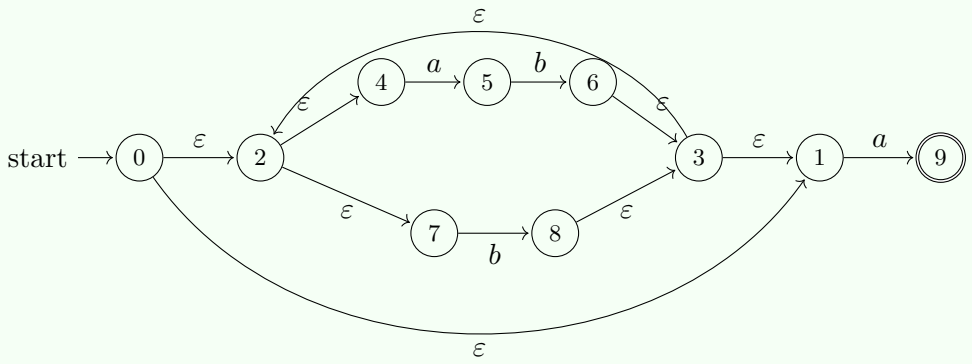
(a) Parse r: give the tree, and count leaves, unions, stars and concatenations.

(b) Build $N(r)$ with convention B. Give every edge, the start and the accepting state, and check the state and edge counts against the formulas.

(c) What are the counts under convention A?

Solution

(a) $r = ((a \cdot b) \mid b)^* \cdot a$: leaves 4 (a, b, b, a), unions 1, stars 1, concatenations 2.
(b) Start 0, accepting 9.



Edges: $0 \xrightarrow{\varepsilon} 2$, $0 \xrightarrow{\varepsilon} 1$ (skip the star), $2 \xrightarrow{\varepsilon} 4$, $2 \xrightarrow{\varepsilon} 7$ (the union), $4 \xrightarrow{a} 5 \xrightarrow{b} 6$ (state 5 is the merge of $N(a)$'s end and $N(b)$'s start), $6 \xrightarrow{\varepsilon} 3$, $7 \xrightarrow{b} 8$, $8 \xrightarrow{\varepsilon} 3$, $3 \xrightarrow{\varepsilon} 2$ (repeat), $3 \xrightarrow{\varepsilon} 1$ (exit), $1 \xrightarrow{a} 9$ (state 1 is the merge of the star's end and $N(a)$'s start). **10 states, 12 edges.** Formulas: $|Q| = 2(4+1+1) - 2 = 10$; $|E| = 4 + 4(1+1) = 12$. (P3) holds: 0, 2 and 3 have two outgoing ε-edges, every other non-final state has exactly one outgoing edge.
(c) Convention A keeps both merged states and joins them by an ε-edge: $2 \cdot 6 = 12$ states, 14 edges.

C4. Subset construction and minimisation (4 points) (⋆⋆)

Using the NFA of C3:

(a) Run the subset construction. Name the DFA states $A, B, C, \ldots$ in the order you discover them and give each one's NFA set.

(b) Minimise the DFA by table filling. Give the final automaton and describe its language in words.

Solution

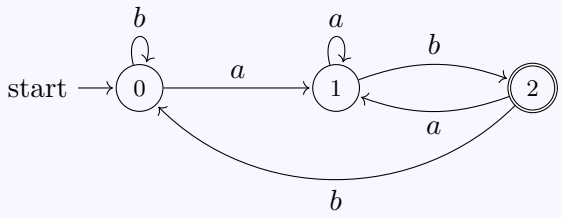
(a) $A = \varepsilon\text{-}closure(\{0\}) = \{0, 1, 2, 4, 7\}$.

State	NFA set	on a	on b
A (start)	$\{0, 1, 2, 4, 7\}$	$\{5, 9\} = B$	$\varepsilon\text{-}closure(\{8\}) = \{1, 2, 3, 4, 7, 8\} = C$
B (accepting)	$\{5, 9\}$	$\emptyset$ (dead)	$\varepsilon\text{-}closure(\{6\}) = \{1, 2, 3, 4, 6, 7\} = D$
C	$\{1, 2, 3, 4, 7, 8\}$	B	C
D	$\{1, 2, 3, 4, 6, 7\}$	B	C

Only B contains 9. Four live states plus the dead state.
(b) Rule 1 marks (A, B), (B, C), (B, D) and (B, dead), and every pair of a live non-accepting state with dead is marked once we notice that A, C, D reach B on a while dead stays dead. Remaining pairs: (A, C), (A, D), (C, D). On a all three go to B, on b all three go to C: nothing distinguishes them, so they merge. Minimal DFA, two states: $X = \{A, C, D\}$ (start) with $X \xrightarrow{a} Y$, $X \xrightarrow{b} X$; $Y = \{B\}$ accepting with $Y \xrightarrow{b} X$, $Y \xrightarrow{a}$ dead.
Language: non-empty strings that end in a and never contain aa (every a except the last is immediately followed by a b). That is exactly $(ab \mid b)^*a$ read right to left.

C5. State elimination, DFA to regular expression (3 points) (⋆⋆)

The DFA M over $\{a, b\}$ has start state 0 and accepting state 2:



(a) Describe $L(M)$ in one sentence.

(b) Apply state elimination (add a new final state, remove 1, then remove 2) and give the resulting expression. Then give the shortest expression you can for the same language.

Solution

(a) Strings that end in ab: state 1 means "the last symbol was a", state 2 "the last two were ab".
(b) Add f with $2 \xrightarrow{\varepsilon} f$. *Remove 1*: in-edges $0 \xrightarrow{a} 1$, $2 \xrightarrow{a} 1$; loop a; out-edge $1 \xrightarrow{b} 2$. New edges $0 \xrightarrow{aa^*b} 2$ and $2 \xrightarrow{aa^*b} 2$. *Remove 2*: in-edge $0 \xrightarrow{a^+b} 2$, loop a^+b, out-edges $2 \xrightarrow{b} 0$ and $2 \xrightarrow{\varepsilon} f$. New edges: $0 \xrightarrow{a^+b(a^+b)^*b} 0$ (joined with the existing loop b) and $0 \xrightarrow{a^+b(a^+b)^*} f$. Result:

$$\big(b \mid a^+b(a^+b)^*b\big)^*\, a^+b\,(a^+b)^*.$$

Shortest: $(a \mid b)^*ab$. Both were checked equivalent on all strings up to length 9. The long form is still correct; state elimination rarely produces the short one, and the exam only requires a correct expression.

C6. Brzozowski derivatives (2 points) (⋆⋆)

Build the DFA of $R = a^*ba^*$ directly by derivatives: compute ∂_a and ∂_b of every expression you reach, say which are nullable, and draw the automaton. Do the same for $(ab)^*$.

Solution

$R = a^*ba^*$: $\partial_a R = (\partial_a a^*)\, ba^* \mid \nu(a^*)\, \partial_a(ba^*) = a^*ba^* \mid \emptyset = R$; $\partial_b R = \emptyset \cdot ba^* \mid \partial_b(ba^*) = a^*$. From a^*: $\partial_a a^* = a^*$, $\partial_b a^* = \emptyset$. Two expressions: R (not nullable, start) and a^* (nullable, accepting), plus $\emptyset$ as the dead state. Transitions: $R \xrightarrow{a} R$, $R \xrightarrow{b} a^*$, $a^* \xrightarrow{a} a^*$, $a^* \xrightarrow{b} \emptyset$. This is the minimal DFA for "exactly one b"; the subset construction on the Thompson NFA gives 2 live states too.
$(ab)^*$: $\partial_a(ab)^* = b(ab)^*$, $\partial_b(ab)^* = \emptyset$; $\partial_b(b(ab)^*) = (ab)^*$, $\partial_a(b(ab)^*) = \emptyset$. States $(ab)^*$ (nullable, start and accepting) and $b(ab)^*$; the DFA is a 2-cycle a then b, everything else dead.

C7. Lower bounds and non-regularity (4 points) (⋆⋆⋆)

(a) Let $L = \{w \in \{a, b\}^* : \#a(w) - \#b(w) \equiv 0 \pmod 3\}$. Show that every DFA for L needs at least 3 states, and give one with exactly 3.

(b) Show with distinguishable strings that $\{a^n b^m : n \geq m\}$ is not regular.

(c) Show with the pumping lemma that the palindromes over $\{a, b\}$ are not regular.

Solution

(a) ε, a, aa are pairwise distinguishable: suffix ε separates $\varepsilon \in L$ from $a \notin L$; suffix a separates ε ($a \notin L$) from aa ($aaa \in L$); suffix bb separates a (abb: $1 - 2 = -1 \notin$) from aa ($aabb \in L$). So 3 states at least. The DFA with states $0, 1, 2 = (\#a - \#b) \bmod 3$, a adding 1 and b subtracting 1 (mod 3), start and accept at 0, has exactly 3.
(b) Take a^i and a^j with $i < j$ and the suffix b^j: $a^j b^j \in L$ but $a^i b^j \notin L$. Infinitely many pairwise distinguishable strings, so no finite DFA.
(c) Suppose palindromes are regular with pumping constant p. Take $w = a^p b a^p \in L$, $|w| \geq p$. Any split $w = uvz$ with $|uv| \leq p$, $|v| \geq 1$ has $v = a^j$, $j \geq 1$, inside the first block. Then $uv^2z = a^{p+j}ba^p$, which reads differently backwards ($a^p b a^{p+j}$), so it is not in L. Contradiction.

C8. The combined DFA and a scan with pushback (4 points) (⋆⋆⋆)

Token rules, in priority order: `P1 = a`, `P2 = ab+`, `P3 = b*a`.

(a) Build the combined DFA (one automaton for all three rules) and label every accepting state with the token it reports.

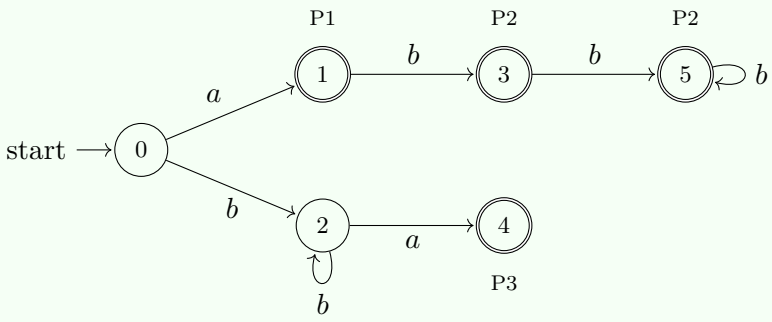
(b) Which accepting state's label is decided by rule order rather than by length? Explain.

(c) Scan `abbabaa` with longest match. For each token give the states visited, the last accepting position, the pushback, and the token. How many characters are read in total?

(d) What does the scanner do on the input `bbb`?

Solution

(a)



State 0: start. State 1 (read `a`): accepting for `P1` and for `P3` (with zero b's). State 2 (`b`$^+$): not accepting, waiting for an `a`. State 3 (`ab`) and 5 (`ab`$^{2+}$): `P2`. State 4 (`b`$^+$`a`): `P3`. Six live states.
(b) State 1: `a` is matched by both `P1` and `P3` with the same length, so the earlier rule `P1` labels it. Every other label is unique.
(c)

Token	Reads (states)	Last accept	Emit, push back
1	a:1(P1), b:3(P2), b:5(P2), a:stuck	after 3, P2	**P2** `abb`, push back a
2	a:1(P1), b:3(P2), a:stuck	after 2, P2	**P2** `ab`, push back a
3	a:1(P1), a:stuck	after 1, P1	**P1** `a`, push back a
4	a:1(P1), end of input	after 1, P1	**P1** `a`

Result **P2 P2 P1 P1**, with $4+3+2+1=10$ reads for 7 characters. The stuck character is read and pushed back each time; that is the "+1" in the pushback formula.

(d) From 0, b:2, b:2, b:2, end of input, and no accepting state was ever visited: there is no token starting at position 1. The scanner reports a lexical error, skips one character and retries, so it reports three errors (reads: $3+2+1=6$). Note the difference from pushback: nothing accepted, so there is nothing to give back.

C9. A hand-written lexer (2 points) (⋆⋆)

The C program below is meant to print `NUM`, `ID`, `EQ` (`==`), `ASSIGN` (`=`) and `OTHER` tokens.

```
int c;
while ((c = getchar()) != EOF) {
    if (isdigit(c)) {
        int v = 0;
        while (isdigit(c)) { v = v * 10 + (c - '0'); c = getchar(); }
        printf("NUM(%d) ", v);
    } else if (isalpha(c)) {
        char buf[32]; int n = 0;
        buf[n++] = c;
        while (isalnum(c = getchar())) buf[n++] = c;
        buf[n] = 0;
        printf("ID(%s) ", buf);
    } else if (c == '=') {
        if (getchar() == '=') printf("EQ ");
        else printf("ASSIGN ");
    } else if (c == ' ' || c == '\n') {
    } else printf("OTHER(%c) ", c);
}
```

(a) Give the output for the input `x1=2 y==3+x` and the correct output.

(b) Name the bugs and fix them with `ungetc`.

Solution

(a) Actual (compiled and run): `ID(x1) NUM(2) ID(y) ASSIGN OTHER(+) ID(x).` Correct: `ID(x1) ASSIGN NUM(2) ID(y) EQ NUM(3) OTHER(+) ID(x).`

(b) Three bugs. (1) The number and identifier loops consume the character that ends the token and drop it: after `x1` the `=` is gone, after `2` the blank is gone (harmless, which is why the bug hides). (2) The `=` branch always consumes a second character: on `y==3` the first `=` was already eaten by the identifier loop, the second `=` reads `3` and prints `ASSIGN`, and the `3` is lost. (3) `buf[32]` has no bound check: a 32-character identifier overflows it. Fix: after each loop, `ungetc(c, stdin);`; in the `=` branch, `int d = getchar(); if (d == '=') ... else { ungetc(d, stdin); ... }`; and guard the loop with `&& n < 31`. With the fixes the program prints the correct sequence (verified).

C10. Predict flex (2 points) (⋆⋆)

```
%%
"if"            printf("IF ");
"i"             printf("I ");
[a-z]+          printf("ID(%s) ", yytext);
[0-9]+          printf("INT(%s) ", yytext);
[0-9]+"."[0-9]* printf("FLT(%s) ", yytext);
"."             printf("DOT ");
"=="            printf("EQ ");
"="             printf("AS ");
[ \t\n]         ;
%%
```

(a) Give the output for `if ifx i 3.14 3.  .5 x==1 x=== 7 iff`.

(b) Give the output for `i@f`.

(c) The rule `"i"` is moved below `[a-z]+`. What does `flex` say, and what changes in the output of (a)?

Solution

(a) `IF ID(ifx) I FLT(3.14) FLT(3.)  DOT INT(5) ID(x) EQ INT(1) ID(x) EQ AS INT(7) ID(iff)`. Points to notice: `ifx` and `iff` are identifiers by length; `3.` is a float because the pattern allows zero digits after the point; `.5` is `DOT INT(5)` because no float pattern starts with a point; `===` is `==` then `=`.
(b) `I @ID(f)`: no rule matches `@`, so flex's default rule echoes it to the output unchanged. Add a catch-all `.` rule to turn that into a proper error.
(c) `warning, rule cannot be matched`: for the input `i` both `[a-z]+` and `"i"` match one character and the earlier rule now wins, so the `"i"` rule is dead. In (a), `I` becomes `ID(i)`; nothing else changes (`IF` is still listed first).

C11. Lexer states (2 points) (⋆⋆)

```
%x STR COM
%{
int depth = 0; char buf[256]; int n;
%}
%%
\"            { n = 0; BEGIN(STR); }
<STR>\\n      { buf[n++] = '\n'; }
<STR>\\\"     { buf[n++] = '"'; }
<STR>\\.      { buf[n++] = yytext[1]; }
<STR>\n       { printf("ERR "); BEGIN(INITIAL); }
<STR>\"       { buf[n] = 0; printf("STR(%s) ", buf); BEGIN(INITIAL); }
<STR>.        { buf[n++] = yytext[0]; }
"/*"          { depth = 1; BEGIN(COM); }
<COM>"/*"     { depth++; }
<COM>"*/"     { if (--depth == 0) BEGIN(INITIAL); }
<COM>.|\n     ;
[a-z]+        printf("ID(%s) ", yytext);
[ \t\n]       ;
.             printf("OTHER(%c) ", yytext[0]);
```

```
%%
```

(a) Output for `a "x\"y" /* b /* c */ d */ e`, with the state after each rule fires.

(b) Output for the two lines `p "abc` and `q`.

(c) Delete the rule `<COM>"/*"`. What is the output of (a) now?

(d) Why must `<STR>\\.` come after the two rules above it, and why does `<STR>\n` fire at all given the rule `<STR>.`?

Solution

(a) `ID(a) STR(x"y) ID(e)`. `a`: ID in INITIAL. `"`: to STR. `x`: appended. `\"`: the escaped-quote rule (length 2 beats the `.` rule's length 1), appends a quote. `y`, then `"`: closes, prints, back to INITIAL. `/*`: COM, depth 1. `b`: eaten. `/*`: depth 2. `c`; `*/`: depth 1, stay in COM. `d`; `*/`: depth 0, INITIAL. `e`: ID.
(b) `ID(p) ERR ID(q)`: the newline inside the string triggers the error rule, which also leaves STR so that `q` is lexed normally.
(c) `ID(a) STR(x"y) ID(d) OTHER(*) OTHER(/) ID(e)`: the inner `*/` now closes the comment at depth 1, and `d */` is lexed as code.
(d) All three escape rules match exactly two characters on `\n` or `\"`, so only rule order decides; `\\.` placed first would swallow both specific cases. `.` never matches a newline in flex, so on a newline `<STR>\n` is the only candidate.

C12. A comment regex with a two-character opener and a three-character closer (2 points) ($\star\star\star$)

HTML comments start with `<!--` and end at the first `-->`. Let x stand for any character other than `-` and `>`.

(a) Give a single regular expression for a complete comment.

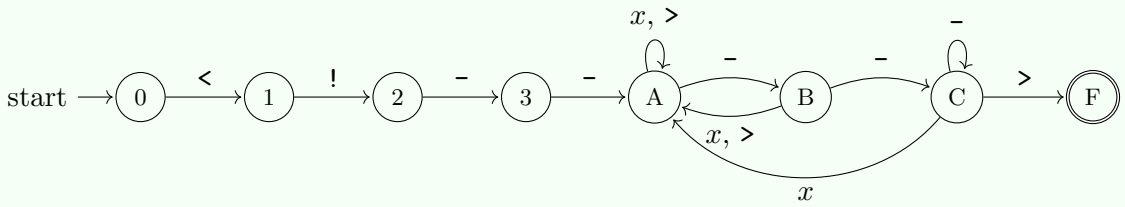
(b) Draw the DFA and say what each state remembers.

Solution

(a) The body may contain `-` and `>` as long as `-->` never appears. Chunks that cannot complete a closer: x, `>`, a single dash followed by x or `>`, or a run of two or more dashes followed by x (a run followed by `>` would close). The closer is two or more dashes then `>`:

$$\texttt{<!--}\ \big(x \mid \texttt{>} \mid \texttt{-}x \mid \texttt{->} \mid \texttt{--}^{+}x\big)^{*}\ \texttt{--}^{+}\texttt{>}$$

(Checked against the DFA below on all strings up to length 9.)
(b) Eight live states.



0 to 3: reading the opener. A: in the body, last character not a dash. B: last character was exactly one dash. C: the last two or more characters were dashes, so `>` now closes. F: done. All other moves go to the dead state. The three body states are needed because the closer is three characters long: the automaton must remember how much of `-->` it has already seen, which is 0, 1 or "at least 2" dashes.

Part D: True or false (20 × 0.5 points)

Answer each statement and give the one-word reason.

D1 to D20

1. A compiler's output must be machine code.
2. Semantic analysis is a separate tree walk because its constraints are not context-free.
3. The lexer decides whether two occurrences of a name refer to the same variable.
4. The parser runs in $O(n^3)$ for every grammar.
5. Every regular language is accepted by some DFA with a unique minimal number of states.
6. Every NFA can be converted to an equivalent DFA.
7. Every DFA with n states accepts a language that some NFA accepts with fewer than n states.
8. ε-closure must be taken after every move in the subset construction.
9. In a DFA, a single failed path means the string is rejected.
10. In an NFA, a single failed path means the string is rejected.
11. Thompson's construction yields an NFA with at most two outgoing edges per state.
12. A regular expression and its minimal DFA may both be exponentially larger than each other, in different cases.
13. Rule order in a lexer specification can override longest match.
14. A lexer with pushback can read $\Theta(n^2)$ characters on an input of length n.
15. `.` in flex matches a newline.
16. Nested block comments can be handled by a regular expression if it is long enough.
17. A keyword that appears inside a comment is still recognised by the keyword rule.
18. Rice's theorem implies that an optimiser can never remove any dead code.
19. Python's indentation-based blocks are lexed with the help of a stack of indentation widths.
20. A minimal DFA may have a dead state.

Answers

1. **False.** Assembly text, bytecode or even another high-level language are all targets.
2. **True.** Declaration-before-use, types, scopes are context-sensitive.
3. **False.** Scoping is semantic analysis; the lexer only produces `ID` with a lexeme.
4. **False.** $O(n^3)$ is the general CFG bound; LL/LR parsers for the grammars used in practice are linear.
5. **True.** The minimal DFA is unique up to isomorphism.
6. **True.** Subset construction.

7. **False.** For some languages the NFA cannot be smaller (e.g. a^*: one state either way).
8. **True.** Otherwise states reachable by ε are missed.
9. **True.** A DFA has exactly one path.
10. **False.** Reject only when every path fails.
11. **True.** Property (P3): at most two out-edges, and then both are ε.
12. **True.** L_k blows up regex to DFA; state elimination blows up DFA to regex.
13. **False.** Order decides ties of equal length only.
14. **True.** `ab` and `(ab)*c` on $(ab)^k$ gives $k(k+1)$ reads.
15. **False.** `.` excludes newline; `[^x]` includes it.
16. **False.** Unbounded depth is not regular; a counter is needed.
17. **False.** Inside the COMMENT state the keyword rule does not exist.
18. **False.** It implies the optimiser cannot be *exact*; it removes code it can prove dead.
19. **True.** `INDENT`/`DEDENT` are emitted by comparing against a stack of widths.
20. **True.** The dead state is kept when some character has no legal continuation (it is usually drawn as missing edges).

Where to revise

Questions	Chapter and section
A1–A9, A20, B1–B5, D1–D4, D18	*01 Introduction to Compilers*: §1.2 compilation vs interpretation, §3 the phases and front/back ends, §4 the Chomsky hierarchy, §6 classic questions
A10, A15–A18, C1, C2, C9, D13, D14	*Lexical Analysis Complete*, Part I (§1–§4): tokens, hand-written lexers, token patterns, longest match and rule order
A11–A14, C3–C7, D5–D12, D20	*Lexical Analysis Complete*, Part II: Methods 1 to 7 (Thompson, simulation, subset construction, minimisation, state elimination, derivatives, lower bounds and the pumping lemma)
C8, D14	*Lexical Analysis Complete*, §13–§14: the combined DFA and longest match in a running lexer
A17, A19, C10–C12, D15–D17, D19	*Lexical Analysis Complete*, §16 lexer states, the comment DFA and quirky languages, and §17 flex and JFlex